Δυναμικός προγραμματισμός

Ο δυναμικός προγραμματισμός (ΔΠ) λύνει ένα πρόβλημα σπάζοντάς το σε μικρότερα υποπροβλήματα, λύνοντας το καθένα μία φορά και αποθηκεύοντας τη λύση του για να την ξαναχρησιμοποιήσει.

- Εφαρμόζεται όταν υπάρχουν επικαλυπτόμενα υποπροβλήματα και βέλτιστη υποδομή.
- Βέλτιστη υποδομή: η βέλτιστη λύση του προβλήματος περιέχει βέλτιστες λύσεις των υποπροβλημάτων.

Το παράδειγμα Fibonacci

Η αναδρομή $F(n) = F(n-1) + F(n-2)$ υπολογίζει τα ίδια υποπροβλήματα ξανά και ξανά και έχει εκθετικό χρόνο, περίπου $O(1{,}618^n)$.

```
def fib(n, memo={}):
    if n < 2:
        return n
    if n not in memo:
        memo[n] = fib(n - 1) + fib(n - 2)
    return memo[n]
```

- Με απομνημόνευση (memoization) κάθε $F(k)$ υπολογίζεται μία φορά: χρόνος $O(n)$.

Top-down και bottom-up

- Top-down: αναδρομή με απομνημόνευση. Υπολογίζει μόνο τα υποπροβλήματα που χρειάζονται.
- Bottom-up: γεμίζουμε πίνακα από τα μικρότερα υποπροβλήματα προς τα μεγαλύτερα, χωρίς αναδρομή.

```
def fib(n):
    a, b = 0, 1
    for _ in range(n):
        a, b = b, a + b
    return a
```

- Η bottom-up έκδοση του Fibonacci χρειάζεται μόνο O(1) μνήμη, γιατί κρατά τις δύο τελευταίες τιμές.

Πρόβλημα του σακιδίου 0/1

Έχουμε n αντικείμενα με βάρος w_i και αξία v_i, και σακίδιο χωρητικότητας W. Θέλουμε τη μέγιστη αξία χωρίς να ξεπεράσουμε το W· κάθε αντικείμενο το παίρνουμε ολόκληρο ή καθόλου.

Έστω K[i][w] η μέγιστη αξία με τα πρώτα i αντικείμενα και χωρητικότητα w. Τότε $K[i][w] = \max(K[i-1][w], v_i + K[i-1][w - w_i])$ όταν $w_i \le w$.

- Χρόνος και μνήμη O(n·W). Ο χρόνος είναι ψευδοπολυωνυμικός, γιατί εξαρτάται από την τιμή του W.

Μέγιστη κοινή υπακολουθία (LCS)

Για δύο συμβολοσειρές X και Y ψάχνουμε τη μεγαλύτερη υπακολουθία που εμφανίζεται και στις δύο (όχι απαραίτητα συνεχόμενα). Χρησιμοποιείται π.χ. στο diff αρχείων.

Αν X[i] = Y[j], τότε L[i][j] = L[i−1][j−1] + 1· αλλιώς L[i][j] = max(L[i−1][j], L[i][j−1]).

- Χρόνος και μνήμη O(|X|·|Y|).

Πρόβλημα	Υποπρόβλημα	Χρόνος
Fibonacci	F(k), k < n	O(n)
Σακίδιο 0/1	K[i][w]	O(n·W)
LCS	L[i][j]	O(|X|·|Y|)

Σύνοψη

- ΔΠ = υποπροβλήματα + αποθήκευση λύσεων, ώστε να μη λύνονται ξανά.
- Χρειάζεται επικαλυπτόμενα υποπροβλήματα και βέλτιστη υποδομή.
- Top-down με απομνημόνευση ή bottom-up με πίνακα.
- Κλασικά παραδείγματα: Fibonacci, σακίδιο 0/1, LCS.

Για την επόμενη φορά: λύστε το πρόβλημα των ρέστων (ελάχιστος αριθμός κερμάτων) με ΔΠ.